Astronomy 465 – Fall 2024

Lab # 2: Characterizing a radio telescope - noise and temperature calibration

Reports due: Friday Oct 25, 11 pm via Canvas

Purpose: As explained in Chapters 3 and 4 of the "Fundamentals of radio astronomy" textbook, radio astronomers express power measured from astronomical sources using the equivalent temperature (called antenna temperature). The noise power produced by the various receiver components, as well as the background sky noise temperature (the CMB radiation, synchrotron radiation etc), is called the system temperature T_{sys} and determines the rms fluctuation for a spectra-line observation. Therefore, T_{sys} represents a very important characteristic of any radio telescope.

As described in Section 4.1.2 of the "Fundamentals of radio astronomy", we can measure T_{sys} using the noise diode with temperature T_{cal}. This is done using a switching experiment where the diode injects power into the telescope's receiver – we can label this measurement V_{cal}. If we look at a blank sky and measure the voltage when the diode is turned off, this measurement is V_{off}. Since T_{cal} is known from the specifications of the diode being used, we can estimate T_{sys} via:

$$T_{sys} = \frac{V_{off}}{V_{cal} - V_{off}} T_{cal}. \quad (1)$$

This calculation is exactly what the SRT performs when you run the calibration function *noisecal*. Two spectra are obtained, V_{off} and V_{cal}, and the resultant T_{sys} value is saved in the .rad file. This calculation, however, relies on the knowledge of T_{cal}. If T_{cal} is wrong, as could be due to the noise diode deteriorating over time, our estimate of T_{sys} will be wrong. (Currently, both SRTs assume $T_{cal} = 103$ K.)

For this lab, to check how well are noise diodes performing, we will estimate T_{sys} in a different way and compare our results with the value calculated by the SRT software. We will also compare our HI spectral-line observations with the published Leiden-Dwingeloo Survey (LDS) survey.

This lab has 3 parts: (1) Run the noise diode (*noisecal*) to calculate the system temperature of the SRT; (2) obtain an HI spectrum for a position in the Galactic plane and compare your result with a spectrum from a published HI survey – this is effectively providing an absolute calibration for our measurements; (3) estimate T_{sys} by measuring the rms noise in your obtained HI spectrum and compare this estimate with what the SRT software reported; (4) OPTIONAL - test how close is the SRT receiver to an ideal radiometer.

Observing script: Prepare a command file that does the following:

- Open a data file, set the frequency to 1420.4 MHz (this is the frequency of the 21-cm line of atomic hydrogen) in mode 4 (this gives 156 frequency bins centered at 1420.4 MHz with a spacing of 0.0078125 MHz).

- Select a position in the Milky Way plane preferably in the outer Galaxy (Galactic longitude 90 to 270 degrees). Apply an Elevation offset of 30 degrees to get to a position off the Galactic plane. Run the calibration diode (*noisecal*) there. Then drive back to your selected position in the Galactic plane. Record observations in a unique file name. Integrate for 10-15 minutes.

Procedure: Run your observing script during the lab period, or at a later time.

For this lab we will use our observations obtained with the SRT but also calibrated data (organized in a spectral line data cube) of the 21-cm emission from the whole northern Milky Way obtained by one of professional HI surveys - the Leiden-Dwingeloo Survey (LDS).

The LDS data cube, called "LDSfull.fits" is available as a fits data file on the class Canvas page[1] .

To access and open FITS LDS data cube in Python, please see the last section. Once you read the LDS data cube, you will be able to extract HI spectra at desired positions.

Analysis:

1. Read the SRT data file in Python using the strategy you developed in Lab 1. Note in your .rad file the estimated T_{sys} value provided by the SRT software.

2. **Produce the averaged HI spectrum:** Average all HI spectra at your selected position in the Galactic plane. The average HI spectrum is in units of antenna temperature. Using information in your observing data file, calculate the frequency array that will be plotted on the x-axis. Make a plot of your averaged HI spectrum as a function of frequency. Use the non-relativistic Doppler shift equation to convert frequency into Topocentric Radial Velocity. Make a plot of the Antenna Temperature vs. Topocentric Radial Velocity.

   Perform a linear polynomial fit to frequency channels with no obvious HI emission to model the spectral baseline; subtract the baseline from your spectrum. Make a plot of the baseline subtracted averaged HI spectrum as a function of Topocentric radio velocity.

3. **Extract an HI spectrum from the LDS survey:** Access the LDS FITS data cube and locate the position of your observations. As LDS has a higher angular resolution than our SRT, to compare spectra at the same resolution, average 12 x 12 LDS spectra around your selected position. Make a plot, this will show HI Brightness temperature as a function of Radial Velocity.

   How does the peak of the HI spectrum compare between your observations and LDS? If our calibration was correct, those two spectra should compare well. Find the scaling

[1]To visually examine the data cube you can use astronomy packages: ds9 https://sites.google.com/cfa.harvard.edu/saoimageds9 or CARTA https://cartavis.org/.

between the two spectra. In other words, find the scaling we would need to apply to correct our calibration so that our spectrum matches well the LDS spectrum. We can call this scaling the main beam efficiency (please see lecture notes). Also, please comment on how the two spectra compare in terms of their overall shapes.

4. **New estimate of T_{sys}:** Back to your baseline-subtracted averaged HI spectrum. Apply the scaling from the previous step so that your spectrum now agrees reasonably well with the LDS data.

   We can now use the radiometer equation to estimate T_{sys}. For a range of frequency channels away from the HI peak which look essentially like noise only, measure the rms noise of your baseline-subtracted average HI spectrum. To estimate the total integration time used to produce this spectrum use the following: the real integration time is about 1/15 of the actual elapsed time during your observation (yes, the SRTs take a lot of time to write data and the real observing time is far less than what we requested!). The bandwidth of a single frequency channel is 0.0078125 MHz. You now have everything you need to estimate T_{sys} by using the radiometer equation.

   Compare your estimated T_{sys} with the value from Step 1 that was recorded by the SRT software. What can you conclude? Are those two measurements close, or very different?

5. **OPTIONAL - Radiometer:** to check how close our SRT is to an ideal radiometer, for your position produce an average HI spectrum by using a different number of scans. For example, if you had 100 scans in total for this position, investigate HI spectra obtained by averaging 5 scans, then 10 scans, 50 scans, etc, until you have included all scans. Each produced HI spectrum should be multiplied by the scaling the main beam efficiency constant. Then measure the rms noise in each spectrum (using channels without any HI emission) and plot this as a function of the number of scans used for integration (e.g. 5, 10, etc. - of course this is equivalent to integration time spent to observe particular spectrum). Compare this plot with the relation we expect for an ideal radiometer. What can you conclude?

6. Prepare an individual lab report starting with an introduction, then explaining your observing and data processing strategy, describing and plotting results, and conclusions. Discuss the principal sources of uncertainty in your estimate of T_{sys}, both random and systematic.

1 How to read a FITS data cube

```
import numpy as np
import matplotlib.pyplot as plt
import astropy.units as u
import os
from astropy.io import fits

file='/Users/snez/LDSfull.fits'
cube = fits.open(file)
data = cube[0].data
print(data.shape)
# this will save the data portion for the FITS file
# dimensions are: (389, 361, 720); 389 values along the velocity axis, 361 values alo

# to extract and plot a single spectrum at pixel position (120, 150):
spectrum = data[:, 120, 150]
plt.plot(spectrum)

# to get the header information from the FITS file.
# the header contains info about coordinate axes which is important
# for search for the right (l, b) position in the cube.
header = cube[0].header
print(header)
cube.close()

## Read from the header velocity axis information:
velocity_center = header["CRVAL3"]/1000.
velocity_inc = header["CDELT3"]/1000.
velocity_pix = header["CRPIX3"]

#have as many velocity channels as are in the spectrum
channels = np.arange(len(spectrum))

#define velocity as km/s blocks
velocity = velocity_center + velocity_inc * (channels-velocity_pix)
velocity = np.array(velocity, dtype = float)
plt.plot(velocity,spectrum)
```